

Senior Full Stack Developer

Technical Assessment

Expense Claim API

Time Allowance	Approximately 4 hours (you have 1 week from receipt to submit)
Tech Stack	Java 21, Spring Boot (recent stable version), relational database of your choice
Deliverables	GitHub repository containing source code, README, and database schema
Follow-up	You will walk through your solution during the interview.

1. Scenario

You are building a backend API for an internal expense claim system. Employees can submit expense claims, and approvers can approve or reject them.

Your API will be consumed by a frontend application (which you do not need to build). Focus entirely on the backend: API design, data access, security, and code quality.

We have deliberately kept this brief short on implementation detail. How you choose to solve each requirement is as important to us as whether you solve it. Be prepared to explain your technical decisions during the walkthrough.

2. Requirements

2.1 Users & Access Control

The system has two roles: EMPLOYEE and APPROVER. Users must authenticate before accessing the API. The application should start with the following seed users pre-loaded:

Username	Password (for testing)	Role
john.smith	Password123!	EMPLOYEE
jane.doe	Password456!	EMPLOYEE
mike.approver	ApproverPass1!	APPROVER

These passwords are provided in plaintext for testing purposes only. How they are stored is up to you.

2.2 Expense Claims

1. An employee can submit a new expense claim containing: a description (max 255 characters), an amount, the date the expense was incurred, and a category (one of: TRAVEL, MEALS, ACCOMMODATION, EQUIPMENT, OTHER).
2. An employee can view their own claims. They should not be able to see other employees' claims.
3. An approver can view all pending claims across all employees.
4. An approver can approve or reject a pending claim. When rejecting, a reason must be provided. A claim that has already been approved or rejected cannot be actioned again.

2.3 Audit Trail

Every state change on an expense claim (submission, approval, rejection) must be recorded with enough information to answer the question: “who did what, and when?” An approver should be able to retrieve the audit history for a given claim.

3. What We Care About

The following are the areas we will evaluate during the walkthrough:

Area	We'll Be Looking At
Security	How you handle authentication, authorisation, password storage, input handling, and protection of sensitive data. We will probe the choices you've made and the threats you've considered.
API Design	Whether the API is well-structured, uses appropriate HTTP semantics, and returns meaningful responses — including when things go wrong.
Data Layer	Your schema design, how you ensure data integrity, and whether your data access patterns are efficient for the expected query load.
Code Structure	How you organise your code, manage dependencies, and apply design principles. We value clarity over cleverness.
Testing	That you test meaningfully. We want to see tests that give us confidence the application works correctly, including at least one test that exercises a realistic end-to-end flow.
Documentation	A README that lets us build, run, and test your application with minimal friction. If we can't get it running, we can't assess it.

4. Submission

- Push your solution to a GitHub repository (public or private) and share the link.
- Include your database schema as a SQL file.

- Ensure the application builds and runs cleanly from the instructions in your README.
- Do not over-engineer. Two hours is the target, please respect your own time. If you run short, document what you would have done next; we value that honesty.

5. A Few Notes

- There is no single correct answer. We are interested in your reasoning as much as your code.
- You are free to use any OSS libraries or tools you see fit. If you use AI coding tools, that's fine, but be prepared to explain every line of your solution as if you wrote it yourself.
- If anything in this brief is ambiguous, make a reasonable assumption and document it. Good judgement under ambiguity is something we value.

Good luck! We look forward to reviewing your solution.